

# 袁志刚

男 | 21岁 | Java后端开发 (具备AI Agent开发经验) | 2027届  
☎ 17731930675 ✉ weiqiang0322@gmail.com

🌐 <https://github.com/weiqiang612>  
🏠 个人博客(作品集) : weiqiang.me

## 教育背景

华北理工大学 | 计算机科学与技术 (本科) 2023.09 - 2027.07  
GPA 3.59/4.0 (前20%) | CET-6 | 蓝桥杯省赛二等奖

## 专业技能

- **Java & 框架**：深入掌握 Java 核心机制 (集合框架、反射、动态代理)；掌握 Spring Boot、MyBatis，深入理解 IoC、AOP 等核心机制；深入理解常见设计模式 (创建型、结构型、行为型)；能运用 Hutool、Lombok 等工具库提升开发效率。
- **数据存储**：掌握 MySQL 数据库设计与优化，具备 SQL 调优、索引优化、分库分表等实践经验；掌握 Redis 缓存中间件，实践过 Caffeine 多级缓存、分布式锁、缓存穿透/雪崩解决方案、哨兵集群搭建等，具备高并发下的缓存架构设计能力。
- **AI 应用开发**：熟悉 Spring AI，具备 AI Agent (工具调用、记忆系统、多步任务编排) 开发经验及基于 Pgvector 的完整 Hybrid RAG 架构落地实践；熟悉 Vibe Coding、SDD、Harness Engineering 等多种 AI 编程范式；熟练使用 Claude Code / Cursor / Codex 等 AI 编程工具高效开发全栈项目。
- **工程与全栈**：熟练掌握 Linux、Git、Maven、Docker、Nginx；具备前后端分离全栈开发能力 (Vue.js + Spring Boot + Axios)；独立负责过项目的打包部署、运维和上线。

## 项目经历

### 苍穹外卖 AI 智能客服 Agent | Java 后端开发 | 2026.05 - 2026.06

基于 Spring Boot + MyBatis 构建的餐饮 SaaS 系统，在此之上实现了一套完整的 AI 智能客服 Agent，覆盖订单操作、菜品咨询、地址管理等核心场景，支撑日均 1000+ 条咨询的自动化处理。

- **Agent 调度管道**：基于 Spring AI Advisor 链实现意图识别、用户上下文注入、对话记忆、RAG 检索、工具过滤的多层级联调度；根据识别意图动态下发工具白名单 (如购物车意图仅开放 6 个相关工具)，通过调用签名检测与最大轮次限制防止 LLM 陷入工具调用死循环，工具幻觉调用率显著下降。
- **多步任务编排**：识别三类复合意图 (批量取消、先查单再执行、多意图串联)，将"取消最近两个未送达订单"等复杂请求自动分解为有序步骤；通过 Redis 持久化执行状态实现高风险步骤 (取消、退款) 的确认挂起与恢复；设置动态超时预算实现超时自动熔断，系统可用性达 99.5%。
- **用户记忆系统**：对话结束后异步调用 LLM 从用户发言中提取结构化事实 (菜品偏好、口味、饮食限制、地址等) 持久化存储，置信度低于 0.7 自动丢弃；按意图类型以三档粒度将用户自设信息差异化注入上下文 (全量 / 摘要 / 不注入)，支持用户通过 REST API 手动修正并记录审计历史，个性化推荐准确率提升明显。
- **Hybrid RAG 检索**：基于 Pgvector 实现，离线支持 PDF / Markdown / QA / TXT 四种文档类型差异化分块索引 (QA 按问答对切分、Markdown 按标题层级切分)；在线阶段向量检索与关键词全文检索并行执行，经 RRF 融合后送入 Reranker 精排；支持 Query Expansion 自动扩展相关问题提升召回，客服回答准确率达 99.4%。
- **安全限流与缓存优化**：基于 Redisson 滑动窗口 构建 HTTP (AOP 拦截) 与 WebSocket 双通道流控以保护 LLM 算力；在 Advisor 链最前置引入 JVM 内存语义缓存 对 FAQ 提问执行余弦相似度匹配，命中时短路返回以绕过 RAG 与大模型推理；利用 Redis Pub/Sub 广播 在数据变更时触发多实例本地缓存一致性异步重载，问答时延降至毫秒级。
- **后端基础保障**：针对订单状态流转的并发竞争，使用条件更新实现原子写操作；设计"支付回调记录表 + 唯一约束"方案在数据库层拦截重复回调，确保支付幂等性。

### 黑马点评 (高并发场景实战) | Java 后端开发 | 2026.04 - 2026.05

针对社交电商场景下的高频访问与核心交易进行抗压练习，重点解决分布式场景下的缓存一致性、热点发现及超卖问题。

- **多级缓存抗压**：针对热点商铺查询，构建 Caffeine + Redis 二级缓存架构；通过"空值缓存 + 逻辑过期"策略有效规避缓存穿透与击穿；压测下核心接口响应延迟显著降低，彻底阻断高并发流量对数据库的冲击。
- **高并发秒杀优化**：利用 Redis + Lua 脚本实现库存预扣减与用户一人一单的原子化校验，配合 Redisson 分布式锁兜底，彻底解决集群环境下的超卖问题，确保高并发高吞吐下数据无误差流转。
- **异步解耦与削峰**：引入 Redis 延迟队列与事件订阅机制，将高耗时的订单超时自动取消及支付后置业务逻辑进行异步解耦，实现瞬时峰值流量削峰，大幅提升系统在高并发瞬间的吞吐量与系统稳定性。